Coding Assessment: Production-Ready AI Chat Application

Role: Software Engineer (full stack) | Assignment: Build and deploy a ChatGPT-like application using React, FastAPI, PostgreSQL, and AWS.

1. Functional Requirements

Build an AI-powered chat application supporting:

- Chat interface: Send messages and receive AI-generated responses using an AI API of your choice.
- Conversation management: Create, view, rename, and delete conversations.
- Chat history: Persist conversations and messages in PostgreSQL.
- User authentication: Secure login with user-specific conversation history.
- User experience: Responsive UI, loading states, error handling, and streaming responses (preferred).

2. Technical Requirements

Component	Required Technology
Frontend	React.js + TypeScript
Backend	Python + FastAPI (REST APIs)
Database	PostgreSQL
Cloud Hosting	AWS (services of your choice)
CI/CD	GitHub Actions, AWS CodePipeline, or equivalent
Infrastructure as Code	Terraform, AWS CDK, or CloudFormation
Custom Domain	e.g., chat.yourdomain.com (HTTPS required)

3. Deliverables

- GitHub repository containing frontend, backend, tests, IaC, and CI/CD configuration.
- Live application URL accessible through a working custom domain with HTTPS.
- README.md with architecture diagram, local setup, deployment instructions, and design decisions.

4. Additional Guidelines

- You may use AI coding assistants, but must understand and explain your implementation.
- Choose appropriate AWS services and justify architectural trade-offs, cost, security, and scalability.
- Prioritise working, maintainable software over unnecessary complexity.
- Be prepared to explain the architecture and demonstrate a code change deployed via CI/CD.

Expected Outcome: A publicly accessible, secure, functional AI chat application demonstrating end-to-end software engineering and cloud deployment skills.